

Settings refactoring

rydnr

October 15, 2022

Copyright 2017 by rydnr.

The contents of this book are protected under the Creative Commons Attribution-ShareAlike 3.0 Unported license.

You are **free**:

- to **Share**: to copy, distribute and transmit the work,
- to **Remix**: to adapt the work,

Under the following conditions:

Attribution. You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

For any reuse or distribution, you must make clear to others the license terms of this work. The best way to do this is with a link to this web page:

<http://creativecommons.org/licenses/by-sa/3.0/>

Any of the above conditions can be waived if you get permission from the copyright holder. Nothing in this license impairs or restricts the author's moral rights.



Your fair dealing and other rights are in no way affected by the above. This is a human-readable summary of the Legal Code (the full license):

<http://creativecommons.org/licenses/by-sa/3.0/legalcode>

Contents

Illustrations	ii
----------------------	-----------

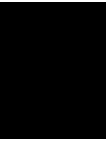
I PharosEDA

1 Introduction	3
1.1 Limitations	3
1.2 Motivation	4
1.3 Objective	4
1.4 Load the project	4
2 Application registry	5
2.1 Registration of EDAAplications	8
3 Different subtrees for each application	9
4 First Ports and Adapters	11
4.1 Minimal behavior shared among all Settings Ports	11
5 Environment setting	13
5.1 Environment-variable adapter	13
5.2 Predefined list adapter	18
5.3 In-memory adapter	20
6 Port resolver	21
6.1 Discovering adapters	21
6.2 A port to choose adapters	23
7 Priorities in adapters	27
7.1 Refactoring EDASettings	28
7.2 Refactoring EDAAplication	29
8 Announcing setting changes	33
8.1 Refreshing SettingBrowser windows	39
8.2 Refreshing the environment adapter	40
Bibliography	43

Illustrations

Part I

PharoEDA



Introduction

PharoEDA is a Pharo framework to simplify the development of event-driven applications, using Event Sourcing, following DDD principles.

At runtime, PharoEDA provides primary adapters for consuming messages and publishing events. Its objective is to take complexity out of the application as much as possible.

It encourages TDD, and also provides a test framework.

1.1 Limitations

Currently, only one PharoEDA application can be running in a Pharo image at the same time.

An `EDAApplcation` has configurable properties, that can be inspected and modified using `SettingBrowser`. The `SettingBrowser` customization is done in `EDASettings` class, using the `<systemsettings>` pragma, which requires the method to be on the class side. Currently, the methods in charge of creating the customization tree expect each setting is stored in specific classes, available via accessors, in class-side attributes. The main benefit of this approach is to make `EDASettings` unaware of the existence of the `EDAApplcation` class. However, this only applies on the surface, because both classes know where the actual setting values are stored.

Additionally, the configuration settings are read from a json file, which gets identified by an environment variable. But that file is only used for the initial values, and its contents are not synchronized with the values of the application after they are modified using `SettingBrowser` or by any other means. Ideally, each setting could be managed differently: either from an environment variable, from a json file, from a global value, and so on.

1.2 Motivation

This refactoring attempts, on one hand, to avoid using classes as setting containers, and sharing them between `EDASettings` and `EDAApplication`. On the other hand, we aim to be able to manage more than one `EDAApplication` instance. Every instance should have its own settings, and its own subtree in `SettingBrowser`. Finally, the third objective consists of providing a way to abstract the holder of each setting so the actual value can be obtained from different sources.

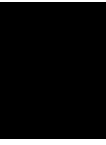
1.3 Objective

This refactoring is divided in different parts. We'll focus first on customizing settings available through `EDAApplication` instances. Then, we'll use `Ports` and `Adapters` to decouple the `EDAApplication` from the actual configuration settings. We'll go through all settings one by one, replacing the current solution with a new adapter. This will enable us to implement new adapters for specific settings. For example, to retrieve credentials from external vaults. At the end, we'll be able to have different `EDAApplication` instances running, each with its own configuration settings, independent of each other.

1.4 Load the project

To load the project,

```
[ Metacello new repository: 'github://rydnr/pharo-eda:settings';  
  baseline: #PharoEDA; load
```



Application registry

We need a way to know which `EDAAApplication` instances exist, and a way to interact with them. A simple registry is enough for now.

Let's start by creating the class with a **registry** attribute we'll use to annotate the references to the existing applications.

```
Object subclass: #EDAAApplications
  uses: EDATLogging
  instanceVariableNames: 'registry'
  classVariableNames: ''
  package: 'EDA-Application'
```

The **registry** should be accessed via dedicated methods, under the *accessing* protocol.

```
EDAAApplications >> registry: aDictionary
  registry := aDictionary
```

```
EDAAApplications >> registry
| result |
result := registry.
result ifNil: [
  result := Dictionary new.
  self registry: result
].
^ result
```

We'll throw an `EDAAApplicationAlreadyRegistered` exception in case the application is already registered.

```
Error subclass: #EDAAApplicationAlreadyRegistered
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Application'
```

Now we can implement a method to register applications. We'll identify each application by its name.

```
EDAAApplications >> register: anEDAAApplication under:
  anApplicationName
self registry
  at: anApplicationName
  ifPresent: [ :app | EDAAApplicationAlreadyRegistered signal: app
    printString ]
  ifAbsentPut: [ anEDAAApplication ]

EDAAApplications >> register: anEDAAApplication
  self
    register: anEDAAApplication
    under: anEDAAApplication applicationName
```

Now each application needs a name, but it could be optional at instance level. We'd like to omit the instance name if only one instance will ever be created. In other words, an instance can have its own name, but can rely upon the class'.

We need to add the new attribute to EDAAApplication:

```
Object subclass: #EDAAApplication
  uses: EDATLogging
  instanceVariableNames: 'applicationName eventStore eventAnnouncer
    commandConsumer commandDispatcher eventProducer
    commandListenerClientDebug isConfigured isStarted
    projectorsByTenant projectionSynchronizationEnabled'
  classVariableNames: ''
  package: 'EDA-Application'
```

The following accessors deal with that:

```
EDAAApplication >> applicationName
  ^ applicationName ifNil: [ self class applicationName ]

EDAAApplication >> applicationName: aString
  applicationName := applicationName
```

We'll delegate on the class itself to provide a default name.

```
EDAAApplication class >> applicationName
  ^ 'default'
```

Now that every application instance has a name, and it's used to reference the application in the registry, we should provide a way to inspect the applications already registered:

```
[ EDASApplications >> edaApplications
  ^ self registry values
```

Recall that EDASApplication was previously meant to be used as a class, not instance. It might be useful to guide developers on how to create new instances. If they use **new** and forget to provide a name, they'll end up with unnamed applications, and won't be able to register more than one in the registry. Therefore, we'll miss precisely the point we're trying to make.

This (class-side) factory method ensures the instances are created with a name. Notice we don't use **self new**. We'll be dealing with that soon enough.

```
[ EDASApplication class >> withName: anApplicationName
  | result |
  result := super new.
  result applicationName: anApplicationName.
  ^ result
```

Analogously, we can create "unnamed" instances as well.

```
[ EDASApplication class >> unnamed
  ^ self withName: self applicationName
```

However, instances can be created without a name (being "unnamed"). If we want to ensure instances can be created only via the factory methods above, we can override the **new** method.

```
[ EDASApplication class >> new
  MessageNotUnderstood
  signal:
    'Use ' , self class printString , ' >> withName: or ' , self
    class printString , ' >> unnamed instead'
```

We'd need to convert EDASApplications into a singleton, to ensure there's only one registry of running applications. To do so, we add a **uniqueInstance** attribute on the class itself.

```
[ EDASApplications class
  uses: EDATLogging classTrait
  instanceVariableNames: 'uniqueInstance'
```

Again, we use a lazy approach in the getter:

```
[ EDASApplications class >> uniqueInstance
  | result |
  result := uniqueInstance.
  result
    ifNil: [
      result := self new.
      self uniqueInstance: result
    ].
  ^ result
```

For the sake of completeness, we define the trivial setter.

```
[ EDAApplications class >> uniqueInstance: anInstance
  uniqueInstance := anInstance
```

2.1 Registration of EDAApplications

At the expense of introducing a cyclic coupling between EDAApplication and EDAApplications classes, it's convenient to make sure all EDAApplication instances get registered automatically.

```
[ EDAApplication class >> withName: anApplicationName
  | result |
  (EDAApplications uniqueInstance edaApplications
   select: [ :app | app applicationName = anApplicationName ])
   isEmpty: [
     result := super new.
     result applicationName: anApplicationName.
     EDAApplications uniqueInstance register: result ]
   ifNotEmpty: [ :c | result := c first ].
  ^ result
```

Finally, we can create applications and get them automatically registered.

```
[ EDAApplication withName: 'test-1'
```

Different subtrees for each application

In `EDASettings` we need to build subtrees dynamically, based on the already registered applications. We can create a new entry using the **group:** method of the `SettingTreeBuilder` that is passed to the `<systemsettings>`-tagged method: `aBuilder group: #subtreeSymbol`

However, the `#subtreeSymbol` we choose must be unique, otherwise the subtree will be created under an existing entry.

Let's remove the current `<systemsettings>` pragma of `EDASettings class>>edaSettingsOn:` and rename that method to `EDASettings class>>edaSettingson:` `aBuilder for: anEDAApplication under: aSymbol`, and let's write a new `EDASettings class>>edaSettingsOn:` from scratch.

Since we haven't created (or registered) any application yet, we can choose either to skip creating anything, or display a message.

```
EDASettings class >> edaSettingsOn: aBuilder
  <systemsettings>
  | root rootSymbol |
  rootSymbol := #edaApps.
  root := aBuilder group: rootSymbol.
  EDAApplcations uniqueInstance edaApplications
    ifEmpty: [
      root
        label: 'No PharoEDA applications' translated;
        description: 'No PharoEDA applications registered' ]
  ifNotEmpty: [ :apps |
    root
      label: 'PharoEDA application(s)' translated , ': ' ]
```

```

        , apps size printString;
        description: 'Registered PharoEDA applications';
        noOrdering.
    apps do: [ :app |
        self edaSettingsOn: aBuilder for: app under: rootSymbol
    ]
]

```

However, the old **edaSettings:** method used a fixed symbol to place the application settings in the tree. Let's fix that. We'll start with the `environmentSettingsOn:under:on:`. The rest of the methods will be fixed later.

```

EDASettings class >> edaSettingsOn: aBuilder for: anEDAApplication
    under: aSymbol
    | parent parentGroup |
    parent := anEDAApplication applicationName.
    parentGroup := aBuilder group: parent.
    parentGroup
        label: parent translated;
        parent: aSymbol;
        description: parent;
        noOrdering.
    self
        environmentSettingsOn: aBuilder
        under: parent
        on: anEDAApplication

```

As you can see, we've renamed the previous method from **environmentSettingsOn: aBuilder under: aSymbol** to **environmentSettingsOn: aBuilder under: aSymbol on: anEDAApplication**. To display and manage each application's settings, we need the application instance. Previously, we used class-scoped attributes of predefined classes as settings. Now we're going to access them through the application instance itself. We need to create the `config/` folder and two files (**development.json** and **integration-tests.json**) with an initial empty json file (`{}`) in them.

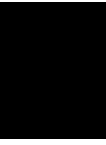
Additionally, we'll comment some settings code in `EDATestSettings` class as well. We'll deal with it later.

```

EDATestSettings class >> edaTestSettingsOn: aBuilder
    <systemsettings>
    (aBuilder group: #edatests)
        label: 'EDA Tests' translated;
        description: 'EDA tests';
        noOrdering;
        parent: #eda

```

We might just add the settings as instance attributes of the `EDAApplication` itself, but such solution would not be flexible enough.



First Ports and Adapters

To accomodate different ways to access and customize setting values, we can abstract them as a Port. `EDApplication` instances will include a reference to a port, regardless of the actual implementation used.

The required ports are: environment, command listener, event publisher, event store, projections, and logging.

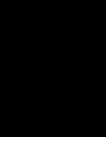
4.1 Minimal behavior shared among all Settings Ports

Even though in Smalltalk there's no such thing as an interface (a definition of the contract exposed by all implementations), we'd like to ensure all adapters share certain behavior. For that, we'll use a trait.

```
[ Trait named: #EDATSettingsPort
  uses: {}
  package: 'EDA-Settings-Ports'
```

The most basic behavior all adapters should support is telling us a brief description of themselves.

```
[ EDATSettingsPort classTrait >> description
  self subclassResponsibility
```

Environment setting

This setting is used to identify the environment. Currently, it shows the value of the **PHARO_ENV** environment variable. The first adapter would be one that implements this behavior.

If we inspect the current logic used in `EDASettings` to inject the environment setting into the `SettingBrowser` tree, we see the following:

```
EDASettings class >> environmentSettingsOn: aBuilder under: aParent
on: app
(aBuilder pickOne: #currentEnvironment)
parent: aParent;
target: EDADUEnvironment;
label: 'Environment' translated;
description: 'Environment';
order: 1;
domainValues:
    (EDADUEnvironment environments
    collect: [ :level | level translated -> level greaseString ])
```

The nature of this setting is to choose one of the available environments. It's mainly informative, but it might impose certain restrictions or influence other settings as well.

5.1 Environment-variable adapter

As its value is currently provided by the **PHARO_ENV** environment variable, it cannot be changed. We'll change the **pickOne:** above with a simple label, as we did before (when there're no registered applications). We can create a new `EDA-Settings-Adapters-EnvVars` package for this kind of adapters.

```
Object subclass: #EDAEnvironmentSettingsEnvVarAdapter
  uses: EDATSettingsPort
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-EnvVars'
```

Now we need to distinguish between the case in which the value is a single string, and the previous assumption that it'd always be a list of items. A simple way to accomplish it is to ask the port itself. Remember that EDASettings relies on the EDAApplication instance to provide the settings, and the latter only deals with ports, not adapters. It makes sense to create a new custom EDATSettingsPort specifically for the environment-related adapters.

```
Trait named: #EDATEnvironmentSettingsPort
  uses: EDATSettingsPort
  package: 'EDA-Settings-Ports'
```

Of course, we'll use that trait in our current adapter instead of the generic one we were using so far.

```
Object subclass: #EDAEnvironmentSettingsEnvVarAdapter
  uses: EDATEnvironmentSettingsPort
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-EnvVars'
```

This variation can now be accommodated in the new EDATEnvironmentSettingsPort.

```
EDATEnvironmentSettingsPort >> supportsMultiple
  self shouldBeImplemented
```

Now, EDASettings can include both variations:

```
EDASettings class >> environmentSettingsOn: aBuilder under: aParent
  on: app
  | entry port |
  port := app environmentSettingsPort.
  (port supportsMultiple)
    ifTrue: [
      entry := aBuilder pickOne: #currentEnvironment.
      entry
        domainValues:
          (EDADUEnvironment environments
            collect: [ :level | level translated -> level
              greaseString ]) ]
    ifFalse: [
      entry := aBuilder setting: #currentEnvironment ].
  entry
    parent: aParent;
    target: port;
```

5.1 Environment-variable adapter

```
label: 'Environment' translated;
description: 'The current environment' translated;
order: 1
```

EDAApplication has to provide an adapter for its **environmentSettingsPort** somehow. We'll deal with a mechanism to choose from a list of available adapters soon. For now, let's return the one we've just implemented: **EDAEnvironmentSettingsEnvVarAdapter**.

```
Object subclass: #EDAApplication
  uses: EDATLogging
  instanceVariableNames: 'applicationName commandConsumer
    commandDispatcher commandListenerClientDebug
    environmentSettingsPort eventAnnouncer eventProducer eventStore
    isConfigured isStarted projectorsByTenant
    projectionSynchronizationEnabled'
  classVariableNames: ''
  package: 'EDA-Application'.
```

```
EDAApplication >> environmentSettingsPort
| result |
result := environmentSettingsPort.
result ifNil: [
  result := EDAEnvironmentSettingsEnvVarAdapter new.
  self environmentSettingsPort: result
].
^ result
```

```
EDAApplication >> environmentSettingsPort: anAdapter
anAdapter = environmentSettingsPort
ifTrue: [ ^ self ].
environmentSettingsPort := anAdapter
```

When we launch a **SettingBrowser** we get an **SubclassResponsibility** error. Let's implement it in our adapter. But does it support a list of values? Actually, no. We cannot change the value of the environment value.

```
EDAEnvironmentSettingsEnvVarAdapter >> supportsMultiple
^ false
```

Running again **SettingBrowser**, it will try to call **currentEnvironment** on our adapter, but we haven't implemented it yet. Let's do it now.

```
EDAEnvironmentSettingsEnvVarAdapter >> currentEnvironment
^ OSPlatform current environment at: 'PHARO_ENV' ifAbsent:
  '(PHARO_ENV not set)'
```

SettingBrowser should render the tree now, but we're not done yet with this adapter. **EDASettings** assumes the setting can be customized. That is, the adapter supports setting a new value. How can we make it read-only? First of all, **EDASettings** mustn't make any assumption. It's responsibility of the port to provide that information.

So if the port's **isReadOnly** returns **true**, then `EDASettings` will use a `LabelMorph` to display the value provided by the port itself.

We're implementing the ports as traits. It makes sense to indicate all adapters need to override this method.

```
EDATEnvironmentSettingsPort >> isReadOnly
  self shouldBeImplemented
```

Our adapter clearly needs to return **true**:

```
EDAEnvironmentSettingsEnvVarAdapter >> isReadOnly
  ^ true
```

All adapters will share the same entry in `SettingsBrowser`. We'll define that label in `EDATEnvironmentSettingsPort`'s **description**:

```
EDATEnvironmentSettingsPort classTrait >> description
  ^ 'Environment'
```

Based on the port and adapter's metadata, `EDASettings` will choose how to properly display the entry in the tree.

```
EDASettings class >> environmentSettingsOn: aBuilder under: aParent
  on: app
  | entry port |
  port := app environmentSettingsPort.
  port supportsMultiple
    ifTrue: [
      entry := aBuilder pickOne: #currentEnvironment.
      entry
        domainValues:
          (EDADUEnvironment environments
            collect: [ :level | level translated -> level
              greaseString ]) ]
    ifFalse: [
      port isReadOnly
        ifTrue: [
          entry := aBuilder group: #currentEnvironment.
          entry dialog: [ LabelMorph newLabel: port
            currentEnvironment ] ]
        ifFalse: [ entry := aBuilder setting: #currentEnvironment ]
      ].
  entry
    parent: aParent;
    target: port;
    label: port class description greaseString;
    description: port class description greaseString;
    order: 1
```

Refactoring

Even though our adapter is working, it's not reusable. Every other setting whose value comes from an environment variable would need to share most of its behavior. Before continuing with the next adapter, let's refactor this one a bit.

First, let's extract the logic to retrieve values from environment variables into its own trait.

```
[ Trait named: #EDATEnvVarAdapter
  uses: {}
  package: 'EDA-Settings-Adapters-EnvVars'

EDATEnvVarAdapter >> getValueFor: envVarName orElse: defaultValue
  ^ OSPlatform current environment
    at: envVarName
    ifAbsent: defaultValue

EDATEnvVarAdapter >> supportsMultiple
  ^ false

EDATEnvVarAdapter >> isReadOnly
  ^ true
```

Our adapter now uses this new trait instead of EDATSettingsPort:

```
[ Object subclass: #EDAEnvironmentSettingsEnvVarAdapter
  uses: EDATEnvironmentSettingsPort + EDATEnvVarAdapter
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-EnvVars'
```

Now **currentEnvironment** implementation can delegate the logic to the new trait:

```
[ EDAEnvironmentSettingsEnvVarAdapter >> currentEnvironment
  ^ self
    getValueFor: 'PHARO_ENV'
    orElse: '(PHARO_ENV not set)'
```

Additionally, EDAEnvironmentSettingsEnvVarAdapter uses a hard-coded environment variable. Let's fix it as well.

```
[ EDAEnvironmentSettingsEnvVarAdapter >> environmentVariableName
  ^ DUEnvironment environmentVariableName
```

We can refactor the **currentEnvironment** implementation:

```
[ EDAEnvironmentSettingsEnvVarAdapter >> currentEnvironment
  ^ self
    getValueFor: self environmentVariableName
    orElse: '(' , self environmentVariableName , ' not set)',
```

5.2 Predefined list adapter

This adapter allows the user to choose among a predefined list of possible values for the environment setting. From the point of view of EDASettings, if the port supports multiple values, it needs to provide them in advance.

For the new adapter, let's create it under a new package EDA-Settings-Adapters-InMemory.

```
Object subclass: #EDAPredefinedEnvironmentSettingsInMemoryAdapter
  uses: EDATEnvironmentSettingsPort + EDAPrintOnHelper +
    EDATCollectionHelper
  instanceVariableNames: 'currentEnvironment'
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-InMemory'

EDAPredefinedEnvironmentSettingsInMemoryAdapter >> currentEnvironment
^ currentEnvironment ifNil: [ self availableEnvironments first ]
```

It needs to store the selected value in memory.

```
EDAPredefinedEnvironmentSettingsInMemoryAdapter >>
  currentEnvironment: aByteString
  currentEnvironment := aByteString
```

The list of predefined values is provided by the **availableEnvironments** method.

```
EDAPredefinedEnvironmentSettingsInMemoryAdapter >>
  availableEnvironments
^ OrderedCollection with: 'dev' with: 'test' with: 'PRO'
```

We need to tell that the new adapter supports multiple values, so EDASettings can use **pickOne:** to render the setting.

```
EDAPredefinedEnvironmentSettingsInMemoryAdapter >> supportsMultiple
^ true
```

EDASettings can now display the options regardless of the adapter used.

```
EDASettings class >> environmentSettingsOn: aBuilder under: aParent
on: app
| entry port |
port := app environmentSettingsPort.
port supportsMultiple
ifTrue: [
  entry := aBuilder pickOne: #currentEnvironment.
  entry
    domainValues: (
      port availableEnvironments
        collect: [ :level | level translated -> level
greaseString ]) ]
ifFalse: [
```

```

    port isReadOnly
      ifTrue: [
        entry := aBuilder group: #currentEnvironment.
        entry dialog: [ LabelMorph newLabel: port
currentEnvironment ] ]
      ifFalse: [ entry := aBuilder setting: #currentEnvironment ]
    ].
  entry
    parent: aParent;
    target: port;
    label: port class description greaseString;
    description: port class description greaseString;
    order: 1

```

Also, even though now it's not strictly necessary, this adapter supports read-write values, in case anyone needs to know it.

```

[ EDAPredefinedEnvironmentSettingsInMemoryAdapter >> isReadOnly
  ^ false

```

Now, to test it we can just change EDAApplcation to use it instead of the current one. We'll introduce a mechanism to inject adapters into ports dynamically soon.

```

[ EDAApplcation >> environmentSettingsPort
  | result |
  result := environmentSettingsPort.
  result
    ifNil: [
      result := EDAPredefinedEnvironmentSettingsInMemoryAdapter new.
      self environmentSettingsPort: result ].
  ^ result

```

The only missing piece is to provide a description. It'd be great if We'll use some methods from existing traits in PharosEDA.

```

[ Object subclass: #EDAPredefinedEnvironmentSettingsInMemoryAdapter
  uses: EDATEnvironmentSettingsPort + EDAPrintOnHelper +
    EDATInMemoryAdapter + EDATCollectionHelper
  instanceVariableNames: 'currentEnvironment'
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-InMemory'
]
[ EDAPredefinedEnvironmentSettingsInMemoryAdapter class >> description
  ^ 'Predefined list of environments'

```

Refactoring

As we did with the environment-variable adapter, let's introduce a new intermediary trait, common to all "in-memory" adapters, to indicate **isReadOnly** and "supportsMultiple" are **false** for this kind of adapters.

```

[ Trait named: #EDATInMemoryAdapter
  uses: EDATSettingsPort
  package: 'EDA-Settings-Adapters-InMemory'
[ EDAEnvironmentSettingsInMemoryAdapter >> isReadOnly
  ^ false
[ EDAEnvironmentSettingsInMemoryAdapter >> supportsMultiple
  ^ false
[ Object subclass: #EDAPredefinedEnvironmentSettingsInMemoryAdapter
  uses: EDATInMemoryAdapter
  instanceVariableNames: 'currentEnvironment'
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-InMemory'

```

5.3 In-memory adapter

Another useful choice is to let the user decide the name of the environment, and store it in memory.

```

[ Object subclass: #EDAEnvironmentSettingsInMemoryAdapter
  uses: EDATEnvironmentSettingsPort + EDAPrintOnHelper +
    EDATInMemoryAdapter
  instanceVariableNames: 'currentEnvironment'
  classVariableNames: ''
  package: 'EDA-Settings-Adapters-InMemory'

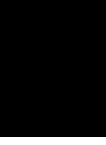
```

We just need to provide the accessors for the attribute, and we're done.

```

[ EDAEnvironmentSettingsInMemoryAdapter >> currentEnvironment
  ^ currentEnvironment ifNil: [ '' ]
[ EDAEnvironmentSettingsInMemoryAdapter >> currentEnvironment: anObject
  currentEnvironment := anObject

```



Port resolver

6.1 Discovering adapters

We have implemented two adapters so far. One that takes the **environment** information from an environment variable, and another one that lets the user choose among a predefined list of values. Which one is used gets determined by `EDAApplication` accessor for **environmentSettingsPort**. How can we decide which adapter should be used? We need a piece to handle that for us.

From `EDAApplication` point of view, given a port, we can find the classes using it. We assume the port is a trait and the classes using it are the adapters the user can choose from. However, we can soften the last assumption by asking the adapter if it accepts the port semantics.

Let's create a new trait `EDATPortResolver`.

```
[ Trait named: #EDATPortResolver
  uses: {}
  package: 'EDA-Settings-Ports'

[ EDATPortResolver >> availableAdapters: aPort
  ^ aPort traitUsers select: [ :cls | cls accepts: self ]
```

By default, `EDATEnvironmentSettingsPort` should reject itself as adapter for any port. It's useful when we have unfinished adapters and we don't want them to interfere with `SettingBrowser`.

```
[ EDATEnvironmentSettingsPort classTrait >>accepts: aPort
  ^ false
```

Our existing adapters can override that method, so they can be discovered by

EDATByPriorityAdapter.

```
[ EDAEnvironmentSettingsEnvVarAdapter class >> accepts: aPort
  ^ true

[ EDAPredefinedEnvironmentSettingsInMemoryAdapter class >> accepts:
  aPort
  ^ true
```

We can move on, and implement the way ports get resolved. First, let's create a new class EDAPortResolver.

```
[ Object subclass: #EDAPortResolver
  uses: EDATPortResolver
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings-Ports'
```

Second, make it be able to provide all available adapters for a given port:

```
[ EDAPortResolver >> resolveSettingsPort: aPort
  | candidates candidate |
  candidates := (self availableAdapters: aPort) sorted.
  candidate :=
    candidates
      ifEmpty: [ nil ]
      ifNotEmpty: [ :col | col first ].
  ^ candidate ifNil: [ nil ] ifNotNil: [ :c | c new ]
```

Third, make EDAAApplication use it.

```
[ EDAAApplication >> environmentSettingsPort
  | result |
  result := environmentSettingsPort.
  result
    ifNil: [
      result := EDAPortResolver new resolveSettingsPort:
        EDATEnvironmentSettingsPort.
      self environmentSettingsPort: result ].
  ^ result
```

If we try to open SettingBrowser, it might fail because the EDAAApplications registry refers to EDAAApplication instances with old code. If that's the case, you can empty the registry and register a new instance.

```
[ EDAAApplications uniqueInstance registry: Dictionary new.
  EDAAApplication withName: 'test-2'.
```

Our naive implementation sorts the adapters it finds, sorts them, and picks the first one. But we can now use it to resolve any other port, not only the **environment's**, by finding all available adapters and sorting them.

6.2 A port to choose adapters

Our purpose now is to avoid the limitation of not being able to select anything but the first adapter. Let's add a new entry in `SettingBrowser` to let us choose among the available adapters. What we are doing for **environment** will be the basis for the rest of the configuration entries.

We need a new **portSettingsPort** attribute in `EDAApplication`:

```
Object subclass: #EDAApplication
  uses: EDATLogging
  instanceVariableNames: 'applicationName commandConsumer
    commandDispatcher commandListenerClientDebug
    environmentSettingsPort eventAnnouncer eventProducer eventStore
    isConfigured isStarted projectorsByTenant
    projectionSynchronizationEnabled portSettingsPort'
  classVariableNames: 'ProjectionSynchronizationEnabled'
  package: 'EDA-Application'
```

Do not forget to generate the accessors as well.

```
EDAApplication >> portSettingsPort
  ^ portSettingsPort

EDAApplication >> portSettingsPort: anObject
  portSettingsPort := anObject
```

The purpose of this port is to let the user choose the adapter (for the other ports) from a list of available options.

Let's define the port itself:

```
Trait named: #EDATPortSettingsPort
  uses: EDATSettingsPort
  package: 'EDA-Settings-Ports'
```

This new port also needs to provide **isReadOnly** and **supportsMultiple** information, required when building the settings tree for `SettingBrowser`. Let's move them from `EDATEnvironmentSettingsPort` to `EDATSettingsPort`, so they are defined for any port from now on.

```
EDATSettingsPort >> isReadOnly
  self shouldBeImplemented

EDATSettingsPort >> supportsMultiple
  self shouldBeImplemented
```

The adapter of this new port will store the selected adapter in memory, but since there'll be no other adapters, so for the sake of clarity we'll call it `EDA-PortSettingsPort`:

```
Object subclass: #EDAPortSettingsPort
  uses: EDATPortSettingsPort + EDAPrintOnHelper
  instanceVariableNames: 'environmentSettingsResolverPortClass'
  classVariableNames: ''
  package: 'EDA-Settings-Ports'
```

Initially, we'll use it to choose the adapter in charge of the **environment** setting. That's why we added it the **environmentSettingsResolverPortClass** attribute.

We need its accessors as well. The setter is straightforward.

```
EDAPortResolverInMemoryAdapter >>
  environmentSettingsResolverPortClass: anAdapterClass
environmentSettingsResolverPortClass := anAdapterClass
```

For the getter, we'll initialize it with **EDAEnvironmentSettingsInMemoryAdapter**.

```
EDAPortResolverInMemoryAdapter >>
  environmentSettingsResolverPortClass
environmentSettingsResolverPortClass
  ifNil: [ self environmentSettingsResolverPortClass:
    EDAEnvironmentSettingsInMemoryAdapter ].
  ^ environmentSettingsResolverPortClass
```

Now we can fix the **EDAApplication** **portSettingsPort** getter to create a new **EDAPortSettingsPort** the first time.

```
EDAApplication >> portSettingsPort
  ^ portSettingsPort
    ifNil: [ | result |
      result := EDAPortSettingsPort new.
      self portSettingsPort: result.
      result ]
```

We are close to be able to manage which adapter to use for our **environment** setting. This new method in **EDASettings** will create the subtree for us.

```
EDASettings class >> portSettingsOn: aBuilder under: aParent on: app
  | entry values key portResolver |
  key := aParent , 'ports'.
  entry := aBuilder group: key.
  entry
    label: 'Ports' translated;
    parent: aParent.
    portResolver := EDAPortResolver new.
    values :=
      OrderedCollection
        withAll:
          ((portResolver availableAdapters:
            EDATEnvironmentSettingsPort)
            collect: [ :adapterClass | adapterClass description
```

6.2 A port to choose adapters

```
translated -> adapterClass ]).  
(aBuilder pickOne: #environmentSettingsResolverPortClass)  
  label: 'Environment adapter' translated;  
  target: app portSettingsPort;  
  parent: key;  
  domainValues: values
```

We need only to call it as part of **edaSettingsOn:for:under:.**

```
EDASettings class >> edaSettingsOn: aBuilder for: anEDAApplication  
  under: aSymbol  
  | parent parentGroup |  
  parent := anEDAApplication applicationName.  
  parentGroup := aBuilder group: parent.  
  parentGroup  
    label: parent translated;  
    parent: aSymbol;  
    description: parent;  
    noOrdering.  
  self  
    environmentSettingsOn: aBuilder  
    under: parent  
    on: anEDAApplication.  
  self portSettingsOn: aBuilder under: parent on: anEDAApplication
```

If we open SettingBrowser now, we can see the new subtree. However, it's not working as we expected. It doesn't seem to display the list of available adapters. Additionally, the only entry is not useful to distinguish the actual adapter.

Let's inspect the adapters EDAPortResolver is able to detect:

```
#EDAPortResolver is missing, and does not understand #new
```

We see it finds only the EDAEnvironmentSettingsEnvVarAdapter. Let's fix its **description** first.

```
EDAEnvironmentSettingsEnvVarAdapter class >> description  
  ^ 'Environment variable'
```

What is missing in order to enable EDAEnvironmentSettingsInMemoryAdapter is overriding the **accepts:** method.

```
EDAEnvironmentSettingsInMemoryAdapter class >> accepts: aPort  
  ^ true
```

Additionally, EDAPredefinedEnvironmentSettingsInMemoryAdapter didn't use the EDATEnvironmentSettingsPort trait:

Priorities in adapters

The list of adapters visible to `EDAPortResolver` are not sorted in any way. The straightforward way to sort them is by using **`collect:sorted:`**.

In the *sorted* block, we need to define how we want them sorted. To avoid unnecessary coupling, we'll use `<` to find out if a given adapter should appear before another. To provide a common way to compare adapters, let's introduce the concept of **`**`**.

The idea is two fold: to define a way to sort adapters, and also a way to implement which adapter should be used by default. Keep in mind this is meant to only compare adapters fitting in a common port.

Let's make sure all adapters implement it.

```
[ EDATSettingsPort classTrait >> priority
  self shouldBeImplemented
```

Now we can use it to compare adapters with `<`, `>`, `<=`, `>=`.

```
[ EDATSettingsPort classTrait >> < anotherAdapterClass
  ^ self priority < anotherAdapterClass priority
```

```
[ EDATSettingsPort classTrait >> > anotherAdapterClass
  ^ anotherAdapterClass priority < self priority
```

```
[ EDATSettingsPort classTrait >> <= anotherAdapterClass
  ^ self priority = anotherAdapterClass priority
  or: [ self < anotherAdapterClass ]
```

```
[ EDATSettingsPort classTrait >> >= anotherAdapterClass
  ^ self priority = anotherAdapterClass priority
  or: [ anotherAdapterClass < self ]
```

Now we can present them sorted by priority (the "Environment adapter" setting under the Port section of each application).

```
portSettingsOn: aBuilder under: aParent on: app
| entry values key portResolver |
key := aParent , 'ports'.
entry := aBuilder group: key.
entry
  label: 'Ports' translated;
  parent: aParent.
portResolver := EDAPortResolver new.
values := OrderedCollection
  withAll:
    ((portResolver availableAdapters: EDATEnvironmentSettingsPort)
     collect:
       [ :adapterClass | adapterClass description translated ->
         adapterClass ])
    sorted: [ :a :b | a < b ]).
(aBuilder pickOne: #environmentSettingsResolverPortClass)
  label: 'Environment adapter' translated;
  target: app portSettingsPort;
  parent: key;
  domainValues: values
```

7.1 Refactoring EDASettings

EDASettings is maybe too clever regarding how to get the list of available adapters. Let's make EDAPortResolver hide that instead.

```
availableEnvironmentSettingsAdapters
^ self availableAdapters: EDATSettingsPort
```

EDASettings now gets a little bit simpler.

```
portSettingsOn: aBuilder under: aParent on: app
| entry values key portResolver |
key := aParent , 'ports'.
entry := aBuilder group: key.
entry
  label: 'Ports' translated;
  parent: aParent.
portResolver := EDAPortResolver new.
values := OrderedCollection
  withAll:
    ((portResolver availableEnvironmentSettingsAdapters
     collect:
       [ :adapterClass | adapterClass description translated ->
         adapterClass ])
    sorted: [ :a :b | a < b ]).
(aBuilder pickOne: #environmentSettingsResolverPortClass)
```

```

label: 'Environment adapter' translated;
target: app portSettingsPort;
parent: key;
domainValues: values

```

Additionally, we can move the sorting logic to the portResolver.

```

availableAdapters: aPort
  ^ (aPort traitUsers select: [ :cls | cls accepts: self ]) sorted

```

EDASettings now can be unaware of any sorting.

```

portSettingsOn: aBuilder under: aParent on: app
  | entry values key portResolver |
  key := aParent , 'ports'.
  entry := aBuilder group: key.
  entry
    label: 'Ports' translated;
    parent: aParent.
  portResolver := EDAPortResolver new.
  values := OrderedCollection
    withAll:
      (portResolver availableEnvironmentSettingsAdapters
        collect:
          [ :adapterClass | adapterClass description translated ->
            adapterClass ]).
  (aBuilder pickOne: #environmentSettingsResolverPortClass)
    label: 'Environment adapter' translated;
    target: app portSettingsPort;
    parent: key;
    domainValues: values

```

7.2 Refactoring EDAApplication

Currently, EDAApplication has a reference to the *environmentSettingsPort*. Recall this is not the same as the *environment*, but the instance able to provide the *environment* value. EDAApplication will delegate which adapter is used in every port to its *portSettingsPort*.

```

portSettingsPort
  ^ portSettingsPort
  ifNil: [ | result |
    result := EDAPortSettingsPort new.
    self portSettingsPort: result.
    result ]

portSettingsPort: anAdapter
  portSettingsPort := anAdapter

```

EDASettings doesn't need to know how adapters are resolved. It just need to use them to build the settings tree.

```
environmentSettingsOn: aBuilder under: aParent on: app
  ^ self
    settingOn: aBuilder
    named: #currentEnvironment
    labelled: 'Environment'
    withDescription: 'Environment'
    usingConfig: app environmentSettingsPort
    under: aParent
    on: app
```

Let's recall what we are doing again. Each EDAAApplication needs certain configuration settings. Instead of storing those values in attributes in certain specific classes, they're now different for each EDAAApplication. However, they're not stored directly in the EDAAApplication instance, but in adapters.

EDASettings retrieves the list of EDAAApplication via the EDAAApplications registry. For each application, it builds a configuration tree. Each configurable entry is managed by the adapters accessible to the EDAAApplication via its *portSettingsPort*. Additionally, the *portSettingsPort* itself, which stores the references of the adapters used, has its own section in the Setting-Browser.

For the environment setting, EDASettings asks the EDAAApplication to provide the value, sending it the *environmentSettingsPort* message. EDAAApplication acts as an intermediary and delegates it to its *portSettingsPort*.

```
environmentSettingsPort
  ^ self portSettingsPort environmentSettingsPort

environmentSettingsPort: anAdapter
  anAdapter = self portSettingsPort environmentSettingsPort
  ifTrue: [ ^ self ]
  ifFalse: [ self portSettingsPort environmentSettingsPort:
    anAdapter ].
```

We now need to store the adapter in EDAPortSettingsPort, in *environmentSettingsPort*, and provide accessors for it.

```
Object subclass: #EDAPortSettingsPort
  instanceVariableNames: 'environmentSettingsPort'
  classVariableNames: ''
  package: 'EDA-Settings-Ports'

environmentSettingsPort
  ^ environmentSettingsPort

environmentSettingsPort: anAdapter
  environmentSettingsPort := anAdapter
```

However, `EDAPortSettingsPort` needs to provide a default adapter. Otherwise `EDASettings` will fail when it tries to build the setting tree. Since all adapters can now be sorted by priority, `EDAPortSettingsPort` can use that criteria to provide one the first time.

```
[environmentSettingsPort
  ^ environmentSettingsPort ifNil: [
    | result |
    result := EDAPortResolver new resolveEnvironmentSettingsPort.
    self environmentSettingsPort: result.
    result
  ]
```

There's one missing piece to implement. `EDASettings` builds a subtree to let the user choose which class of adapter used in each port, in its *portSettingsOn:under:on:* method. The actual option sends the message `#environmentSettingsAdapterClass` to *app portSettingsPort*. So `EDAPortSettingsPort` needs to keep track of the `**` of the adapter used too.

```
[Object subclass: #EDAPortSettingsPort
  instanceVariableNames: 'environmentSettingsPort
    environmentSettingsAdapterClass'
  classVariableNames: ''
  package: 'EDA-Settings-Ports'
```

It might appear a bit confusing: we're referring to the class in *environmentSettingsAdapterClass*, but we are using *environmentSettingsPort* to reference the instance (of the class). We use `**`. It's unaware how they're used elsewhere.

We need to provide accessors, as they are used by `EDASettings`.

```
[environmentSettingsAdapterClass
  ^ environmentSettingsAdapterClass

environmentSettingsAdapterClass := aClass
environmentSettingsAdapterClass := aClass
```

`SettingBrowser` should run fine now.

Announcing setting changes

Now, once an `EDAAApplication` is created, it will access its configuration settings via `portSettingsPort`. If it's not configured via `EDASettings (SettingBrowser)`, `EDAPortSettingsPort` will use `EDAPortResolver` and provide a suitable adapter to store each `EDAAApplication` setting.

What happens if we change the class of the adapter using `SettingBrowser`? Right now, nothing. The `EDAAApplication` doesn't notice. In other words, it doesn't have any effect. Well, `EDAPortSettingsPort` will refer to an instance in `environmentSettingsPort` which won't necessarily match the `environmentSettingsAdapterClass` class.

Later, we'll extend the approach we're following with the `environment` setting to the rest of the configurable parameters. Instead of hard-wiring what to do to the adapters when the `PortSettingPort` adapter classes change, we can use `Announcements`. In the future we'll refactor further, and use `PharoEDA` events.

Let's start by creating a new generic event.

```
[ Announcement subclass: #EDASettingsChanged
  instanceVariableNames: 'oldValue newValue edaApplication'
  classVariableNames: ''
  package: 'EDA-Application'
```

Let's Pharo create the accessors for us.

```
[ oldValue
  ^ oldValue

[ oldValue: anObject
  oldValue := anObject
```

```

[ newValue
  ^ newValue
[ newValue: anObject
  newValue := anObject
[ edaApplication
  ^ edaApplication
[ edaApplication: anApplication
  edaApplication := anApplication

```

It's a good habit to provide a factory method as well.

```

[ old: anOldValue new: aNewValue on: anApplication
  ^ self new
  oldValue: anOldValue;
  newValue: aNewValue;
  edaApplication := anApplication;
  yourself

```

We want to emit an event whenever the *environmentSettingAdapterClass* changes. Let's create that class first.

```

[ EDASettingsChanged subclass:
  #EDAEnvironmentSettingsAdapterClassChanged
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Application'

```

At the end of the day, we would like to close and reopen any *SettingBrowser*. Let's add that feature in a trait.

```

[ Trait named: #EDATSettingBrowserHelper
  uses: {}
  package: 'EDA-Traits'

```

The code is pretty rudimentary. It inspects all submorphs existing in the "world" matching *SettingBrowser* class, deletes them, and creates a new one.

```

[ refreshSettingBrowsers
  self currentWorld submorphs
  select: [ :w | w model class = SettingBrowser ]
  thenCollect: [ :w |
    w delete.
    SettingBrowser open ]

```

We find useful to create a instance method as well.

```

[ refreshSettingBrowsers
  self currentWorld submorphs
  select: [ :w | w model class = SettingBrowser ]
  thenCollect: [ :w |
    w delete.
    .

```

```

[
    SettingBrowser open ]

```

Next thing to do is to write an announcement subscriber, i.e., an instance that gets notified of any EDASettingsChanged event and performs some action in response. In our case, a suitable name could be EDASettingsSubscriber.

```

[
Object subclass: #EDASettingsSubscriber
  uses: EDATSettingBrowserHelper
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Application'

```

This subscriber will receive events of type EDAEnvironmentSettingsAdapter-ClassChanged.

```

[
setupAnnouncer: anAnnouncer application: anEDAApplication
  anAnnouncer
    ifNotNil: [ :announcer |
      announcer
        when: EDAEnvironmentSettingsAdapterClassChanged
        do: [ :event |
          self
            onEnvironmentSettingsAdapterClassChanged: event
            on: anEDAApplication ] ]

```

We now need to refresh any open SettingBrowser windows.

```

[
onEnvironmentSettingsAdapterClassChanged: anEvent on:
  anEDAApplication
  self refreshSettingBrowsers

```

The last piece involves EDAApplication. Whenever the adapter class changes, it'll emit an event. For this, we'll use an Announcer.

```

[
Object subclass: #EDAApplication
  uses: EDATLogging
  instanceVariableNames: 'commandConsumer commandDispatcher
    eventAnnouncer eventProducer eventStore isConfigured isStarted
    projectorsByTenant applicationName portSettingsPort
    environmentSettingsPort settingsAnnouncer'
  classVariableNames: 'ProjectionSynchronizationEnabled'
  package: 'EDA-Application'

```

With its usual accessors.

```

[
settingsAnnouncer
  settingsAnnouncer ifNil: [ self settingsAnnouncer: Announcer new ].
  ^ settingsAnnouncer

settingsAnnouncer: anObject
  settingsAnnouncer := anObject

```

On the other hand, EDAApplication is interested in reacting upon changes in settings. For example, if the port is changed, the actual setting value would

likely change as well. That's precisely the responsibility of the `EDASettingsSubscriber`.

```
Object subclass: #EDAAApplication
  uses: EDATLogging
  instanceVariableNames: 'commandConsumer commandDispatcher
    eventAnnouncer eventProducer eventStore isConfigured isStarted
    projectorsByTenant applicationName portSettingsPort
    environmentSettingsPort settingsAnnouncer settingsSubscriber'
  classVariableNames: 'ProjectionSynchronizationEnabled'
  package: 'EDA-Application'
```

```
settingsSubscriber
  settingsSubscriber
    ifNil: [ self settingsSubscriber: EDASettingsSubscriber new ].
    ^ settingsSubscriber
```

```
settingsSubscriber: anObject
  settingsSubscriber := anObject
```

When an `EDAAApplication` prepares itself, it now needs to set the announcer.

```
setupSettingsAnnouncer
  self settingsAnnouncer: Announcer new
```

Likewise, for setting up the subscriptions.

```
setupSettingsSubscriptions
  self settingsSubscriber
    setupAnnouncer: self settingsAnnouncer
  application: self
```

To ensure the `portSettingsPort` attribute of `EDAAApplication` is initialized properly, let's add a new method.

```
resetPortSettingsPort
  self portSettingsPort: EDAPortSettingsPort new
```

Finally, `EDAAApplication` initialization process should include setting up the announcer and subscriber of settings events.

```
doInitialize
  self isStarted: false.
  self isConfigured: false.
  self resetPortSettingsPort.
  self projectorsByTenant: Dictionary new.
  self setupSettingsAnnouncer.
  self setupEventAnnouncer.
  self setupSettingsSubscriptions
```

We now need to emit an event whenever the class of the *environment* adapter changes. To create an event instance, we need the reference of the associated `EDAAApplication`.

Let's recap for a second the responsibilities of each one:

- **EDAAApplication**: The root of the setting tree, the scope of related settings.
- **EDASettingsSubscriber**: Registers hooks in EDAAApplication whenever settings changes.
- **EDAPortSettingsPort**: Holds which adapter is used for each setting, which is itself a setting.
- **EDASettingsChanged**: The event to emit when a setting changes, which includes the reference to the EDAAApplication.

EDAAApplication holds a reference to EDASettingsSubscriber, but it's only used to add the event listeners, which act back on the EDAAApplication itself. We could pass that responsibility to EDAPortSettingsPort. Actually, let's remove EDASettingsSubscriber altogether (including EDAAApplication accessors).

A feasible design for this problem is to build EDAPortSettingsPort, and any other adapter, with a reference to a new EDASettingsAnnouncer. Its main purpose is to be used to send settings events, and to register subscribers to those events.

```
[ Announcer subclass: #EDASettingsAnnouncer
  instanceVariableNames: 'edaApplication'
  classVariableNames: ''
  package: 'EDA-Settings'
```

The usual accessors:

```
[ edaApplication
  ^ edaApplication

[ edaApplication: anObject
  edaApplication := anObject
```

And a factory method:

```
[ forApp: anEDAAApplication
  ^ self new
    edaApplication: anEDAAApplication;
    yourself
```

EDAAApplication now can create instances of this class, instead of plain Announcer:

```
[ settingsAnnouncer
  settingsAnnouncer
    ifNil: [ self settingsAnnouncer: (EDASettingsAnnouncer forApp:
      self) ].
  ^ settingsAnnouncer
```

Additionally, PortSettingsPort can hold a reference to it as well.

```

Object subclass: #EDAPortSettingsPort
  instanceVariableNames: 'announcer environmentSettingsPort
    environmentSettingsAdapterClass'
  classVariableNames: ''
  package: 'EDA-Settings-Ports'

announcer
  ^ announcer

announcer: anObject
  announcer := anObject

withAnnouncer: anEDASettingsAnnouncer
  ^ self new
    announcer: anEDASettingsAnnouncer;
    yourself

```

We need to skip setting up the settings subscriber in the initialization phase of the EDAApplication, and to remove the method EDAApplication >> setupSettingsAnnouncer we introduced earlier.

```

doInitialize
  self isStarted: false.
  self isConfigured: false.
  self resetPortSettingsPort.
  self projectorsByTenant: Dictionary new.
  self setupEventAnnouncer

```

The next thing to do is to make sure EDAApplication creates EDAPortSettingsPort using the announcer.

```

portSettingsPort
  ^ portSettingsPort
    ifNil: [ | result |
      result := EDAPortSettingsPort withAnnouncer: self
        settingsAnnouncer.
      self portSettingsPort: result.
      result ]

resetPortSettingsPort
  self portSettingsPort: nil

```

EDAPortSettingsPort can now emit a EDAEnvironmentSettingsAdapterClassChanged event whenever the adapter class changes.

```

environmentSettingsAdapterClass: newValue
  | oldValue |
  oldValue := environmentSettingsAdapterClass.
  environmentSettingsAdapterClass := newValue.
  oldValue = newValue
    ifFalse: [ self announcer
      announce:
        (EDAEnvironmentSettingsAdapterClassChanged

```

```

old: oldValue
new: newValue
on: self announcer edaApplication) ]

```

8.1 Refreshing SettingBrowser windows

Now that the event is emitted, we want to react in response. For sure, we want to refresh all SettingBrowser windows.

```

Object subclass: #EDASettingBrowserRefresher
  uses: EDATSettingBrowserHelper
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings'

subscribeToSettingsChangedEventsIn: anEDASettingsAnnouncer
  anEDASettingsAnnouncer
    when: EDASettingsChanged
    do: [ self refreshSettingBrowsers ]

```

We could call this method of EDASettingBrowserRefresher whenever we initialize a new EDASettingsAnnouncer. However, we aim to dynamically discover classes interested in listening to EDASettingsChanged events. Let's do it using traits.

```

Trait named: #EDATSettingsChangedListener
  uses: {}
  package: 'EDA-Settings'

```

At this moment we only expect users of this trait to override the following method:

```

subscribeToSettingsChangedEventsIn: anEDASettingsAnnouncer
  self shouldBeImplemented

```

EDASettingBrowserRefresher can now become user of this trait.

```

Object subclass: #EDASettingBrowserRefresher
  uses: EDATSettingsChangedListener + EDATSettingBrowserHelper
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings'

```

We can now deal with the logic to let users of the trait to subscribe to the events they need.

```

initialize
  super initialize.
  self setupListeners

```

```

[ setupListeners
  EDATSettingsChangedListener traitUsers
  do: [ :l | lsubscribeToSettingsChangedEventsIn: self ]

```

We can check the SettingBrowser windows are refreshed if we reset the applications and create a new application with:

```

[ EDAAApplications uniqueInstance registry: nil
  EDAApplication withName: 'app-1'

```

8.2 Refreshing the environment adapter

If the user changes the adapter class for the *environment* port, the adapter itself should change accordingly. We can model this behavior by calling the *environmentSettingPort*: accessor on EDAApplication, in a separate class.

```

[ Object subclass: #EDAEnvironmentSettingsRefresher
  uses: EDATSettingsChangedListener
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings'

```

The behavior is straight forward: whenever an EDAEnvironmentSettingsAdapterClassChanged is emitted, we want the EDAApplication to use a new adapter instance.

```

[ subscribeToSettingsChangedEventsIn: anEDASettingsAnnouncer
  anEDASettingsAnnouncer
  when: EDAEnvironmentSettingsAdapterClassChanged
  do:
    [ :event | event edaApplication environmentSettingsPort: event
      newValue new ]

```

If you open a SettingBrowser you'll experience a weird behavior. The reason is that EDASettingBrowserRefresher refreshes the SettingBrowser windows before the adapter in EDAApplication is replaced.

A simple test doesn't show any clear behavior regarding the order in which "listeners" are notified.

```

[ announcer := Announcer new.
  announcer when: EDAEnvironmentSettingsAdapterClassChanged do: [
    Transcript print: 'child 1\n' ].
  announcer when: EDASettingsChanged do: [ Transcript print: 'parent
    1\n' ].
  announcer when: EDASettingsChanged do: [ Transcript print: 'parent
    2\n' ].
  announcer when: EDAEnvironmentSettingsAdapterClassChanged do: [
    Transcript print: 'child 2\n' ].
  announcer announce: EDAEnvironmentSettingsAdapterClassChanged.
  Transcript flush

```

Transcript shows the order is not guaranteed.

In this situation, we could either include `EDATSettingBrowserRefresher` trait in all listeners, or make each listener emit a new event after its own processing. The former option is simpler, and we could remove `EDASettingBrowserRefresher`.

```
[Object subclass: #EDAEnvironmentSettingsRefresher
  uses: EDATSettingsChangedListener + EDATSettingBrowserHelper
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Settings'

subscribeToSettingsChangedEventsIn: anEDASettingsAnnouncer
  anEDASettingsAnnouncer
    when: EDAEnvironmentSettingsAdapterClassChanged
    do: [ :event |
      event edaApplication environmentSettingsPort: event newValue
      new.
      self refreshSettingBrowsers ]
```

Voilà. Now it works just fine.

Bibliography

